

提高级 CSP-S 第 13 套初赛模拟试题

一、单项选择题(共 15 题,每题 2 分,共计 30 分;每题有且仅有一个正确选项)

1. 将无符号二进制数 $A = 101010101$ 和 $B = 01010111$ 进行()运算可以使得到的结果最大。
A. $A \text{ and } B$ B. $A \text{ or } B$ C. $A \text{ xor } B$ D. $\sim A$
2. 48 和 60 的最大公约数是()。
A. 1 B. 6 C. 12 D. 24
3. $(2019)_{10} + (2020)_8$ 的结果是()。
A. $(3049)_{10}$ B. $(BF3)_{16}$
C. $(101111110001)_2$ D. $(5765)_8$
4. 当排序的元素值域很小时,使用()最优。
A. 快速排序 B. 归并排序 C. 桶排序 D. 插入排序
5. 下列选项中,()不需要操作系统处理。
A. 管理与配置内存 B. 决定系统资源供需的优先次序
C. 解析 xls 文件 D. 操作网络与管理文件系统
6. 下列选项中,()协议不属于 OSI 七层模型中的应用层(L7)。
A. TCP B. HTTP C. FTP D. SMTP
7. 下列选项中,()不属于解释型语言。
A. Python B. Ruby C. Java D. JavaScript
8. 以下算法中,没有运用贪心思想的是()。
A. Dijkstra 求最短路 B. Huffman Tree 求最优前缀码
C. DP 求合法括号串个数 D. Kruskal 求最小生成树
9. 在一个长度为 n 的数组中,求第 k 大的数(n, k 同阶),最坏情况下最优可以做到()。
A. $O(n \log n)$ B. $O(n)$ C. $O(nk)$ D. $O(n^2)$
10. 5 个有标号球放入 3 个有标号盒子(每个盒子至少放一个)的方案数是()。
A. 540 B. 720 C. 150 D. 144
11. 如果 $x=7, y=7, z=4$,以下代码执行后, val 的值是()。
 $val = 2 + 1 < x/2 \& y + z;$
A. 70 B. 8 C. 10 D. 37
12. 对于一个有 5 个点的完全图(每两个点之间有一条边),至少删去()条边,才能使图中无环。
A. 0 B. 4 C. 5 D. 6
13. 一个双端队列,队中元素顺序为(从队首到队尾)1,2,3,4,5,6,则不可能的出队顺序为()。
A. 1,2,3,4,5,6 B. 1,6,5,3,2,4
C. 1,2,6,3,4,5 D. 6,1,5,4,3,2
14. NOIP 赛场中,符合考场规则的是()。
A. 带入键盘,方便代码编写 B. 窥探他人屏幕
C. 吃方便面 D. 当电脑出现异常时,向监考员示意
15. 下列说法中正确的是()。
A. Dijkstra 是一种求解图上任意边权的单源最短路径的算法
B. 数组可以支持 $O(1)$ 插入一个数
C. BFS 时使用到的数据结构是栈
D. 树中的任意两个点间只有一条简单路径

二、阅读程序(程序输入不超过数组或字符串定义的范围;判断题正确填√,错误填×;除特别说明外,判断题每题 1.5 分,选择题每题 3 分,共计 40 分)

1.

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int n,m,a[15],vis[15];
04 void dfs(int i,int value){
05     if(i==m+1){
06         for(int j=1;j<=m;j++){
07             printf("%d ",a[j]);
08             printf("\n");
09             return;
10         }
11         if(value>n) return;
12         for(int j=value;j<=n;j++){
13             a[i]=j;
14             dfs(i+1,j+1);
15             a[i]=0;
16         }
17     }
18 int main(){
19     scanf("%d%d",&n,&m);
20     dfs(1,1);
21     return 0;
22 }
```

保证题目中输入的 n 和 m 满足 $1 < n < 15$, $0 \leq m \leq n$, 输出的文末换行不占有有效行数。

●判断题

- (1) 当 $m=0$ 时,程序会运行错误。()
 (2) 把 11 行删去后,不会对结果产生任何影响。()
 (3) 当 n 不变, m 增加时,程序输出的行数也会增加。()
 (4) 在程序正常运行的情况下,可能会没有输出。()

●单选题

- (5) 当 $1 < n \leq 6$, $1 \leq m \leq n$ 时,程序的输出行数最大是()。
 A. 6 B. 15 C. 20 D. 24
 (6) 在题目给出的数据范围中,有多少对 (n,m) ,使得程序只输出一行?()。
 A. 1 B. 13 C. 26 D. 28

2.

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int n,m,a[105][105],people[105];
04 int main(){
05     scanf("%d%d",&n,&m);
06     for(int i=1;i<=m;i++){
07         int k,main_people=0;
08         scanf("%d",&k);
09         for(int j=1;j<=k;j++){
```

```

10         scanf("%d",&people[j]);
11     for(int j=1;j<=k;j++)
12         if(people[j]==1) main_people=1;
13     if(main_people){
14         for(int j=1;j<=k;j++)
15             a[people[j]][i]=1;
16     }
17     else{
18         for(int j=1;j<=m;j++){
19             int can=0;
20             for(int l=1;l<=k;l++)
21                 if(a[people[l]][j]) can=1;
22             if(can==1){
23                 for(int l=1;l<=k;l++)
24                     a[people[l]][j]=can;
25             }
26         }
27     }
28 }
29 for(int i=1;i<=n;i++){
30     int ok=1;
31     for(int j=1;j<=m;j++)
32         if(a[1][j]!=a[i][j]) ok=0;
33     if(ok) printf("%d ",i);
34 }
35 return 0;
36 }

```

保证题目中输入 $0 \leq n \leq 100$, $1 \leq m \leq 50$, $0 \leq k \leq n$ 。

● 判断题

- (1) 输出的数的个数不一定小于 n 。()
 (2) 将 18 行的 m 改为 i , 不会影响结果。()
 (3) (2 分) 把 22 行和 25 行删去, 程序的结果会受到影响。()

● 单选题

- (4) 当输入如下时, 输出结果是()。

```

8
5
4 1 3 5 4
2 5 6
3 6 7 8
2 6 2
4 2 6 8 1

```

- A. 1
 B. 1 2 3 8
 C. 1 2 6 8
 D. 1 2 8

- (5) (4 分) 如果 $k=2$, $m=3$, $n=5$, $\text{people}[1]=1$ 且 $\text{people}[2]$ 的输入值是在随机 $[2, 5]$ 的情况下, 那么输出的所有答案的和期望是()。

A. $\frac{39}{32}$

B. $\frac{23}{16}$

C. $\frac{15}{8}$

D. $\frac{6}{5}$

3.

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 const int N=10005,M=10005,P=1e9+7;
04 int n,k,siz[N],dp[N][M][2][2],g[N][M][2][2];
05 vector<int>G[N];
06 void calc(int u,int v,int i,int j){
07     for(int u_flag=0;u_flag<=1;u_flag++)
08         for(int u_covered=0;u_covered<=1;u_covered++)
09             for(int v_flag=0;v_flag<=1;v_flag++)
10                 for(int v_covered=0;v_covered<=1;v_covered++){
11                     if(!v_covered&&!u_flag) continue;
12 int tmp=1ll*dp[v][j][v_flag][v_covered]*dp[u][i][u_flag][u_
covered]%P;
13         g[u][i+j][u_flag][u_covered|v_flag]+=tmp;
14         g[u][i+j][u_flag][u_covered|v_flag]%=P;
15     }
16 }
17 void dfs(int u,int fa){
18     siz[u]=1,dp[u][0][0][0]=1,dp[u][1][1][0]=1;
19     for(auto v:G[u]) if(v!=fa){
20         dfs(v,u);
21         for(int i=min(siz[u],k);i>=0;i--)
22             for(int j=min(siz[v],k-i);j>=0;j--)
23                 calc(u,v,i,j);
24         swap(g[u],dp[u]);
25         for(int i=0;i<=min(siz[u],k);i++)
26             for(int a=0;a<=1;a++)
27                 for(int b=0;b<=1;b++)
28                     g[u][i][a][b]=0;
29         siz[u]+=siz[v];
30     }
31 }
32 int main(){
33     scanf("%d%d",&n,&k);
34     for(int i=2;i<=n;i++){
35         int x,y;
36         scanf("%d%d",&x,&y);
37         G[x].push_back(y),G[y].push_back(x);
38     }
39     dfs(1,0);
40     printf("%d", (dp[1][k][0][1]+dp[1][k][1][1])%P);
41     return 0;
42 }

```

保证题目中输入 $1 \leq n \leq 10^4, 1 \leq k \leq 10^4$, 且输入的边构成一棵树。

● 判断题

- (1) 按照图中存边的方式, 遍历一棵树的最坏时间复杂度是 $O(n^2)$ 。()
 (2) 将 19 行的 `if(v!=fa)` 删去, 不会影响程序的结果。()
 (3) (2 分) 将 21 行, 22 行的 `siz[u]` 和 `siz[v]` 分别替换成 `n` 和 `n`, 不会影响程序运行结果。()

● 单选题

- (4) 当 $n=5, k=3$ 且输入数据构成一条链时, 输出是()。
 A. 0 B. 1 C. 2 D. 3
 (5) (4 分) 当 $n=10^3, k=10^4$ 时, 程序的时间复杂度最接近()。
 A. $O(nk)$ B. $O(n^2)$ C. $O(k^2)$ D. $O(nk^2)$
 (6) (4 分) 当 $n=10^4, k=10^3$ 时, 程序的时间复杂度最接近()。
 A. $O(nk)$ B. $O(n^2)$ C. $O(k^2)$ D. $O(nk^2)$

三、完善程序(单选题, 每小题 3 分, 共计 30 分)

1. 题意简述:

诺风有 n 份资料, 编号为 $1 \sim n$ 。其中第 i 份资料是关于 a_i 号发明的。

诺风想做 m 个统计, 第 i 个统计需要得知第 l_i 到第 r_i 份资料中, 有没有一种发明的资料不少于 $\lceil \frac{r_i - l_i + 1}{2} \rceil$ 份。

如果有, 答案是这种发明的编号, 如果存在两种发明, 请回答编号较小的那一个的编号, 否则答案为 -1。

诺风要去捣鼓他的飞行背包, 所以把问题交给你。

$1 \leq n \leq 2 \times 10^5, 1 \leq m \leq 10^5$

提示:

考虑使用随机算法解决问题, 对每一个问题在询问范围内随机 40 个数, 查询是否符合要求, 可以证明这样单次错误概率在 2^{-40} 。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int i, j, k, m, n, s, t;
04 const int N=2e5+10;
05 struct qry{
06     int rk, l, r;
07 };
08 int a[N], b[N], c[N], ans[N];
09 vector<qry>q[N];
10 vector<int>pos[N];
11 void update(int x, int cc){
12     for(int i=x; i<=n; i+=i&(-i))
13         c[i]+=cc;
14 }
15 int query(int x, int ret=0){
16     for(int i=x; i>=1; i-=i&(-i))
17         ret+=c[i];
18     return ret;
19 }
20 int Rand(int l, int r){
21     return 1ll*rand()*rand()%(r-l+1)+l;
```

```

22 }
23 int main()
24 {
25     memset(ans, -1, sizeof(ans));
26     srand(time(NULL));
27     cin >> n >> m;
28     for(i=1; i<=n; i++)
29         cin >> a[i], b[i] = a[i];
30     sort(b+1, b+n+1);
31     int len = ①;
32     for(i=1; i<=n; i++) {
33         a[i] = lower_bound(b+1, b+len+1, a[i]) - b;
34         ②;
35     }
36     for(i=1; i<=m; i++) {
37         int l, r;
38         cin >> l >> r;
39         if(l==r || l==r+1) {
40             ans[i] = min(a[l], a[r]);
41             continue;
42         }
43         for(j=1; j<=40; j++) {
44             int pos = Rand(1, r);
45             q[a[pos]].push_back((qry){i, l, r});
46         }
47     }
48     for(i=1; i<=len; i++) {
49         for(j=pos[i].size()-1; j>=0; j--)
50             update(pos[i][j], 1);
51         for(j=q[i].size()-1; j>=0; j--) {
52             if(③)
53                 continue;
54             int tmp = query(q[i][j].r) - query(q[i][j].l-1);
55             if(④)
56                 ⑤;
57         }
58         for(j=pos[i].size()-1; j>=0; j--)
59             update(pos[i][j], -1);
60     }
61     for(i=1; i<=m; i++) {
62         if(~ans[i])
63             printf("%d\n", b[ans[i]]);
64         else
65             printf("-1\n");
66     }
67     return 0;
68 }

```

(1) ①处应填写

- A. `unique(b+1,b+n+1)-b` B. `unique(b+1,b+n)-b-1`
C. `unique(b+1,b+n+1)-b-1` D. `unique(b+1,b+n)-b`

(2) ②处应填写

- A. `pos[i].push_back(a[i])` B. `pos[a[i]].push_back(i)`
C. `pos[b[a[i]]].push_back(i)` D. `pos[i].push_back(b[a[i]])`

(3) ③处应填写

- A. `ans[q[i][j].rk]` B. `!ans[q[i][j].rk]`
C. `ans[q[i][j].rk]==-1` D. `~ans[q[i][j].rk]`

(4) ④处应填写

- A. `tmp>=(q[i][j].r-q[i][j].l+1)/2+1`
B. `tmp>=(q[i][j].r-q[i][j].l)/2`
C. `tmp>=(q[i][j].r-q[i][j].l)/2+1`
D. `tmp>=(q[i][j].r-q[i][j].l+1)/2`

(5) ⑤处应填写

- A. `ans[q[i][j].rk]=a[i]` B. `ans[q[i][j].rk]=b[i]`
C. `ans[q[i][j].rk]=i` D. `ans[q[i][j].rk]=b[a[i]]`

2. 题意简述:

话说有一天,天翔哥哥惹到了王若明(叫她平板姬),她十分生气,决定给天翔一点小惩罚,于是她将自己的电脑接入了互联网。

华国的互联网可以看做一张 n 个点 m 条边的无向图,在每条网络通道中穿行需要一定的时间,当然,访问速度越快,花费的时间越少。形式化地说,我们设当前访问的速度为 b ,边权为 a_i ,那么通行这条网络通道需要花费 $\frac{a_i}{b}$ 的时间。

网络中还有一些节点有 k 台不设防的电脑,第 i 台的速度为 vel_i ,王若明一旦到了它们所在的节点就可以直接控制它们(然后就可以一直使用)。

王若明当前的访问速度等于她自己的电脑和她已经控制的电脑的速度之和。

天翔哥哥的电脑在节点 n 处,王若明的电脑在 1 处。王若明想知道,她攻击到天翔哥哥所在的网络节点最少需要花费多长时间。当然,也有可能她不可能攻击到天翔哥哥所在的网络节点,这种情况下,你回答她“Impossible”就可以了。

她知道这是一个 OI 问题,所以她懒得解决,就扔给了你。

王若明还知道这道题的答案是一个浮点数,所以你只需要把答案精确到 10^{-2} 就可以。

$$1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5, k \leq 15$$

输入格式:

第一行四个正整数 n, m, b, k , 代表网络节点数,网络通道数,王若明自己电脑的速度和网络节点中的电脑总数。

接下来 k 行每行两个整数 p_i, x_i , 表示第 i 台电脑处在节点 p_i , 速度为 x_i 。

再下来 m 行每行三个整数 u, v, a , 表示节点 u 到节点 v 有一条边权为 a 的网络通道。

提示:

观察发现我们只在乎存在电脑的位置和起终点位置,所以考虑使用最短路算法处理出两两之间最短路后状压 dp 。

```
01 #include<bits/stdc++.h>
02 #define LL long long
03 using namespace std;
04 struct edge{
```

```

05     LL v, val;
06 };
07 struct com{
08     LL pos, vel;
09 };
10 const LL M=20, N=1e6+10;
11 LL i, j, k, m, n, s, t, now;
12 LL dis[N], vis[N], mp[M][M];
13 double dp[1<<(M+1)][M], vel[1<<(M+1)];
14 double ans=1e12;
15 com c[M];
16 vector<edge>a[N];
17 priority_queue<pair<LL, LL>, vector<pair<LL, LL>>, greater<pair<LL, LL>>>q;
18 bool cmp (com aa, com bb) {
19     return aa.pos<bb.pos;
20 }
21 void dij (LL x) {
22     memset (dis, 1, sizeof (dis)); memset (vis, 0, sizeof (vis));
23     q.push (make_pair (0, x)); dis[x]=0;
24     while (!q.empty()) {
25         LL u=q.top().second;
26         q.pop();
27         if (vis[u])
28             continue;
29         vis[u]=1;
30         for (edge e:a[u]) {
31             if (dis[e.v]<=dis[u]+e.val)
32                 continue;
33             dis[e.v]=dis[u]+e.val;
34             ①;
35         }
36     }
37 }
38 int main() {
39     ans=1e12, now=0;
40     memset (vel, 0, sizeof (vel));
41     cin>>n>>m>>s>>k;
42     for (i=1; i<=n; i++)
43         a[i].clear();
44     for (②)
45         cin>>c[i].pos>>c[i].vel;
46     for (i=1; i<=m; i++) {
47         LL x, y, z;
48         cin>>x>>y>>z;
49         a[x].push_back ((edge) {y, z});
50         a[y].push_back ((edge) {x, z});

```

```

51     }
52     c[1].pos=1,c[1].vel=s,c[k+2].pos=n,c[k+2].vel=0;
53     sort(c+1,c+k+3,cmp);
54     for(i=1;i<=k+2;i++){
55         if(c[i].pos==c[i+1].pos)
56             ③;
57         else
58             c[++now]=c[i];
59     }
60     k=now;
61     for(i=1;i<=k;i++){
62         dij(c[i].pos);
63         for(j=1;j<=k;j++){
64             mp[i][j]=dis[c[j].pos];
65         }
66         if(mp[1][k]>=1e11){
67             printf("Impossible\n");
68             return 0;
69         }
70         for(i=1;i<1<<k;i++){
71             for(j=1;j<=k;j++){
72                 ④;
73                 dp[i][j]=1e12;
74             }
75         }
76         dp[1][1]=0;
77         for(i=1;i<1<<k;i++){
78             for(j=1;j<=k;j++){
79                 if((1<<(j-1))&i)==0)
80                     continue;
81                 for(LL l=1;l<=k;l++){
82                     if(l==j||((1<<(l-1))&i)==0)
83                         continue;
84                     ⑤;
85                 }
86             }
87         }
88         ans=min(ans,dp[i][k]);
89         printf("%.21f\n",ans);
90         return 0;
91     }

```

(1)①处应填写()。

- A. q.push(make_pair(dis[e.v],-e.v))
- B. q.push(make_pair(-dis[e.v],e.v))
- C. q.push(make_pair(dis[e.v],e.v))
- D. q.push(make_pair(-dis[e.v],-e.v))

(2) ②处应填写()。

A. $i=2; i \leq k+1; i++$

B. $i=1; i \leq k; i++$

C. $i=3; i \leq k+2; i++$

D. $i=0; i \leq k-1; i++$

(3) ③处应填写()。

A. $c[now+1].vel = c[i].vel$

B. $c[i].vel = c[i+1].vel$

C. $c[now+1].vel = c[i+1].vel$

D. $c[i+1].vel = c[i].vel$

(4) ④处应填写()。

A. $\text{if}((1 < j) \& i) \text{ vel}[j] += c[i].vel$

B. $\text{if}((1 < (j-1)) \& i) \text{ vel}[j] += c[i].vel$

C. $\text{if}((1 < j) \& i) \text{ vel}[i] += c[j].vel$

D. $\text{if}((1 < (j-1)) \& i) \text{ vel}[i] += c[j].vel$

(5) ⑤处应填写()。

A. $dp[i][1] = \min(dp[i][1], dp[i-(1 < (j-1))][1] + mp[1][j] / vel[i-(1 < (1-1))])$

B. $dp[i][j] = \min(dp[i][j], dp[i-(1 < (j-1))][1] + mp[j][1] / vel[i-(1 < (j-1))])$

C. $dp[i][j] = \min(dp[i][j], dp[i-(1 < (1-1))][j] + mp[j][1] / vel[i-(1 < (1-1))])$

D. $dp[i][1] = \min(dp[i][1], dp[i-(1 < (1-1))][j] + mp[1][j] / vel[i-(1 < (j-1))])$